# RigMaster AI Database Documentation

## 1. components

Description: Stores the hardware catalog used for PC building. Includes CPUs, GPUs, Motherboards, etc. with technical specifications and pricing information in various currencies.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- name: Name of the component (e.g., 'AMD Ryzen 7 7800X3D')
- category: Main hardware category (cpu, gpu, motherboard, etc.)
- sub_category: Specific type (e.g., 'mouse', 'keyboard' for peripherals)
- price/price_usd: Pricing data for budget calculations
- msrp/cost/retail_price: Fallback pricing fields
- currency/currency_code: Currency associated with the stored price
- specs: Object containing technical details (socket, wattage, etc.)
- brand: Manufacturer name
```

## 2. users

Description: Manages user accounts, authentication, and personalized settings.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- username: User's login name
- password: Hashed password string
- email: Contact email address
- is_admin: Boolean flag for administrative access
- preferred_currency: User's selected currency for the UI
- created_at: Timestamp of account creation
```

## 3. saved_builds

Description: Stores PC configurations saved by users, allowing them to revisit or share their builds.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- user_id: Reference to the user who created the build
- build_name: Custom name assigned to the build
- components: Map or list of component IDs used in the build
- total_price: Calculated total price at the time of saving
```

```
- created_at: Timestamp of when the build was saved
- updated_at: Timestamp of last modification
```

## 4. ai_cache

Description: Logs AI requests for analytics and caches responses to improve performance and reduce API costs.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- cache_key: Unique hash derived from the AI prompt (sparse index)
- type: Type of request (assistant, recommend, analysis)
- provider: AI provider used (groq, gemini, etc.)
- response: Cached AI response content
- cached: Boolean indicating if the result was served from cache
- created_at: Timestamp for TTL or logging purposes
```

## 5. shopping_cache

Description: Temporary storage for external shopping results/prices to minimize external API rate limits.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- query: The original search query string (unique index)
- results: List of found parts and prices from external sources
- expires_at: TTL timestamp for automatic deletion by MongoDB
```

## 6. settings

Description: Global configuration for the entire application, manageable via Admin UI.

**Key Fields / Structure:**

```
- _id: Unique identifier (ObjectId)
- key: Unique configuration key (e.g., 'maintenance_mode', 'api_keys')
- value: Corresponding value (can be boolean, string, or complex object)
- description: Internal note about the setting's purpose
```